1️⃣ Deep Learning Fundamentals

1. AI vs ML vs Deep Learning

Artificial Intelligence is the **broad field** of creating machines that can perform tasks requiring human intelligence.

Relationship

Artificial Intelligence
↓
Machine Learning
↓
Deep Learning

Explanation

Concept	Meaning	Example
Artificial Intelligence	Machines that mimic human intelligence	Self-driving cars
Machine Learning	Systems learn patterns from data	Email spam detection
Deep Learning	Uses neural networks with many layers	Face recognition

Real-world Example

- **AI:** Smart assistant answering questions
- **ML:** Predicting house prices
- **DL:** Recognizing faces in photos

---

2. Neural Networks (Simple Explanation)

Definition

A **Neural Network** is a computer model inspired by the human brain that learns patterns from data.

It consists of **connected nodes (neurons)** that process information.

Simple Example

Suppose we want to predict **student marks**.

Study Hours + Attendance → Neural Network → Predicted Marks

The neural network learns the **relationship between inputs and outputs**.

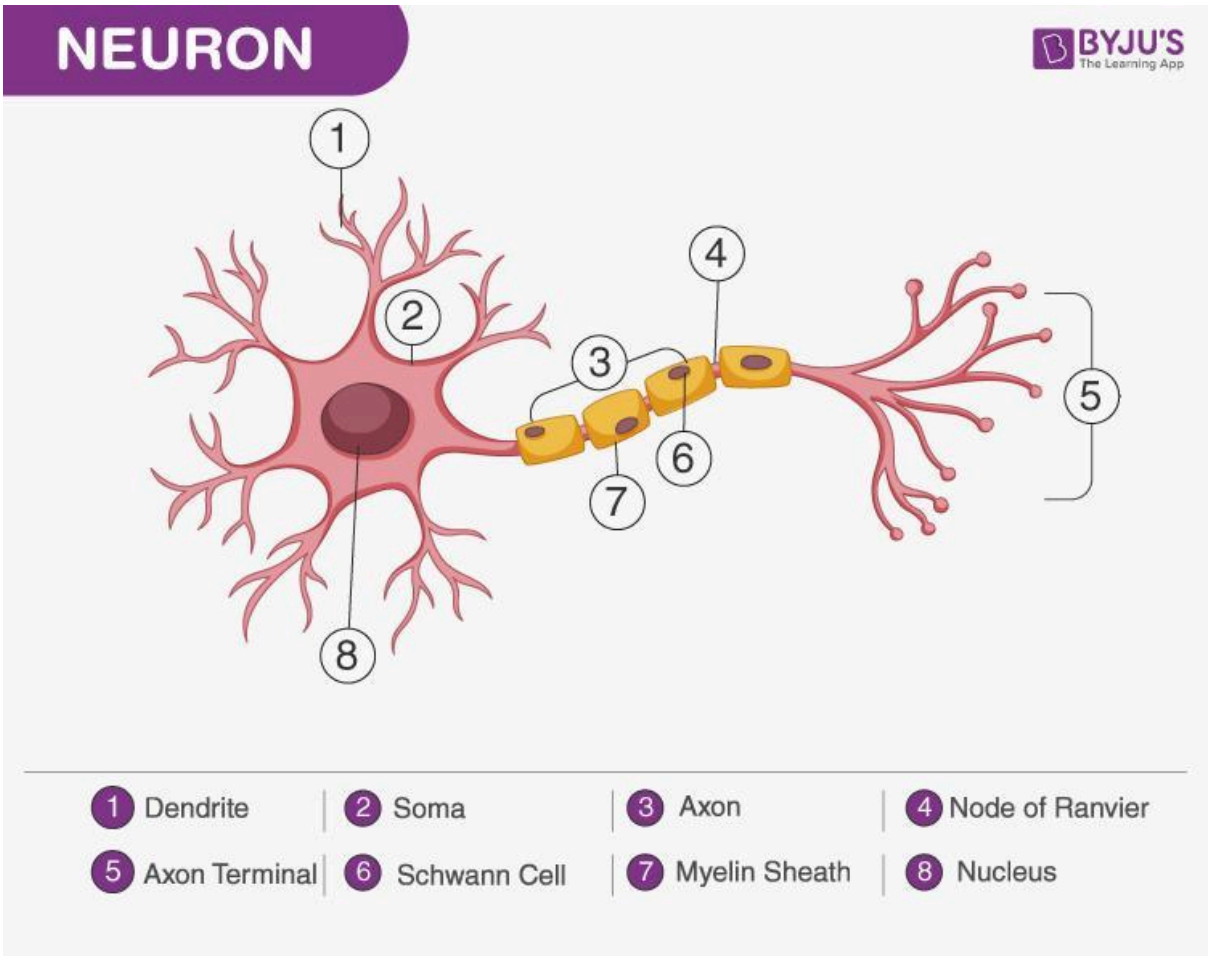
Key Idea

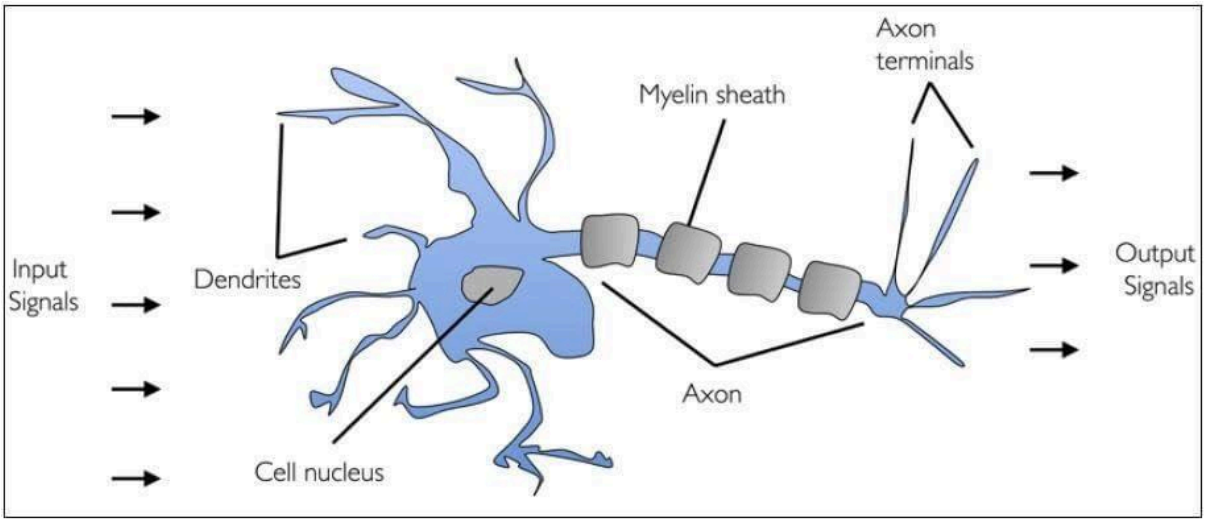
Instead of programming rules manually, the network **learns rules from data**.

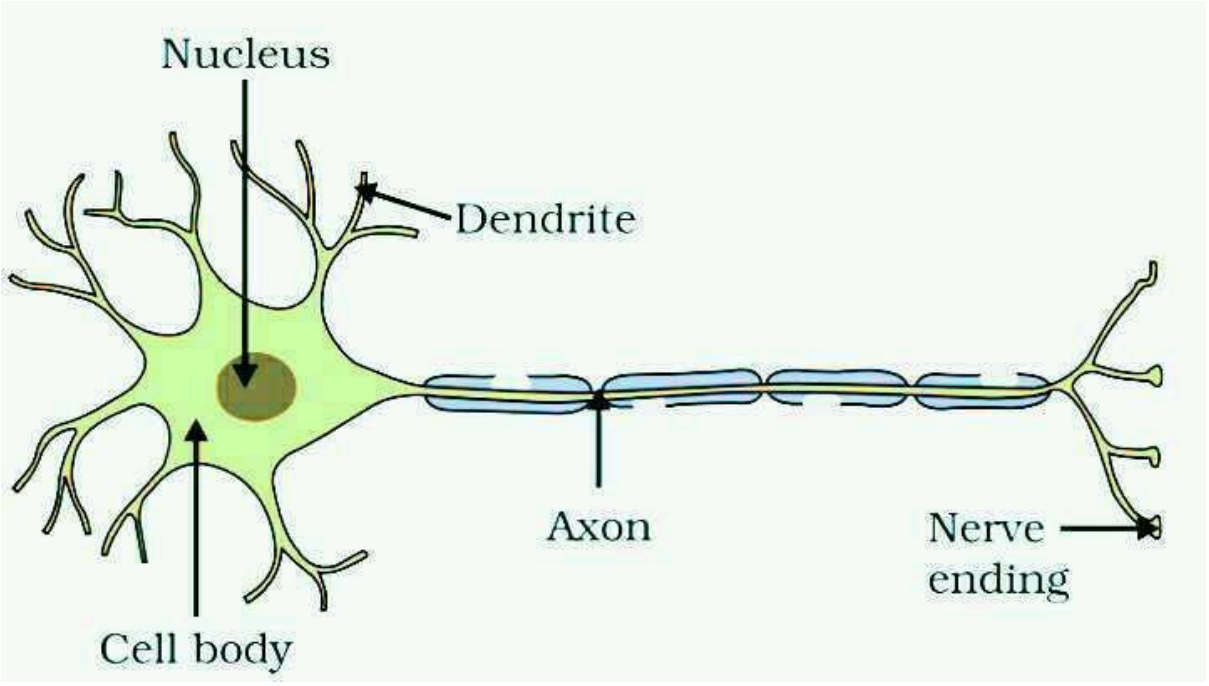
---

3. Biological Neuron vs Artificial Neuron

Biological Neuron







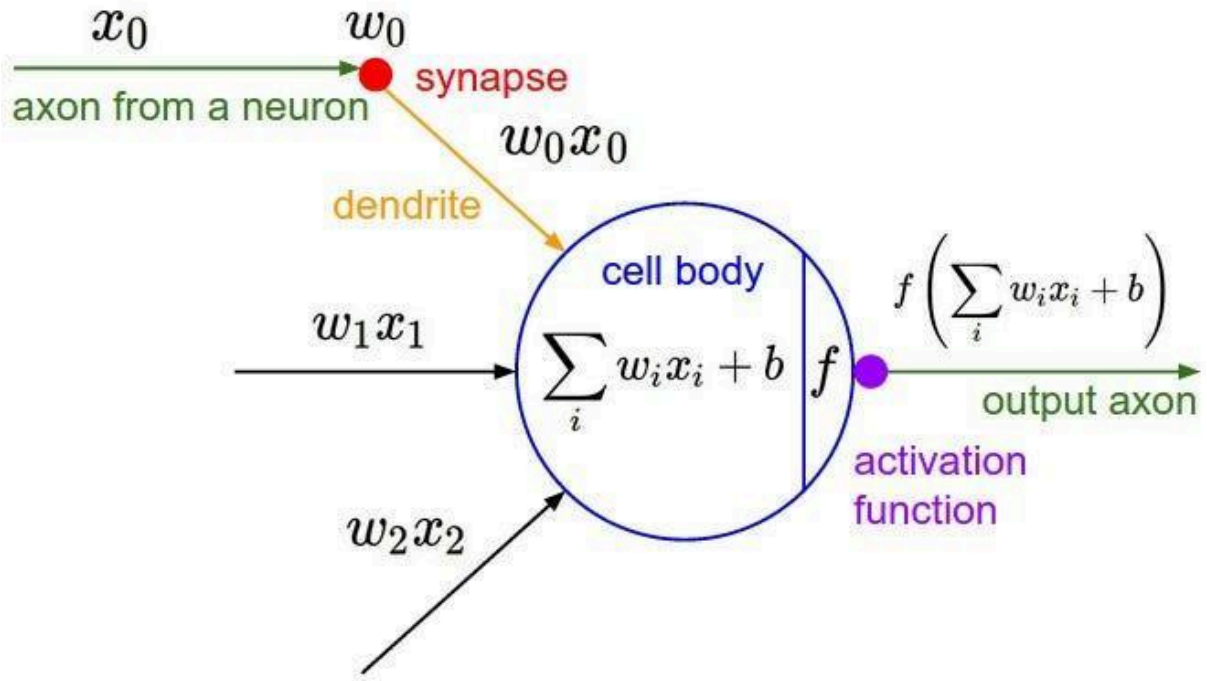
5

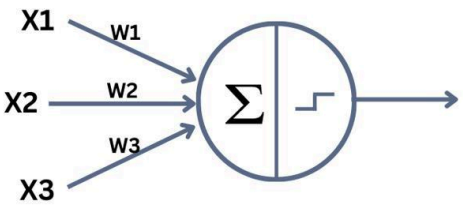
A biological neuron in the human brain has:

Part	Function
Dendrites	Receive signals
Cell Body	Processes signals
Axon	Sends signals to other neurons

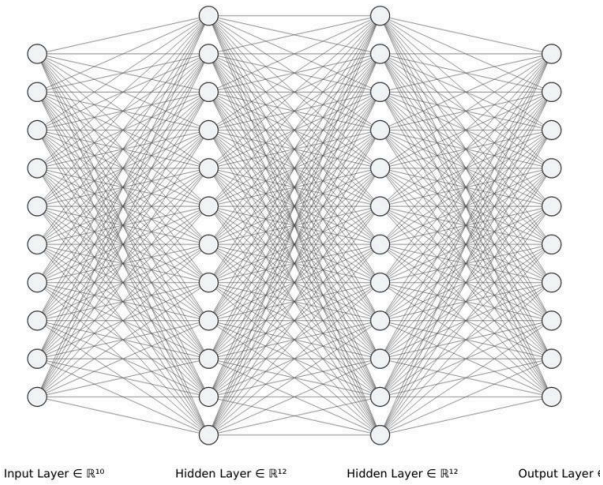
The human brain contains **billions of neurons connected together**.

Artificial Neuron

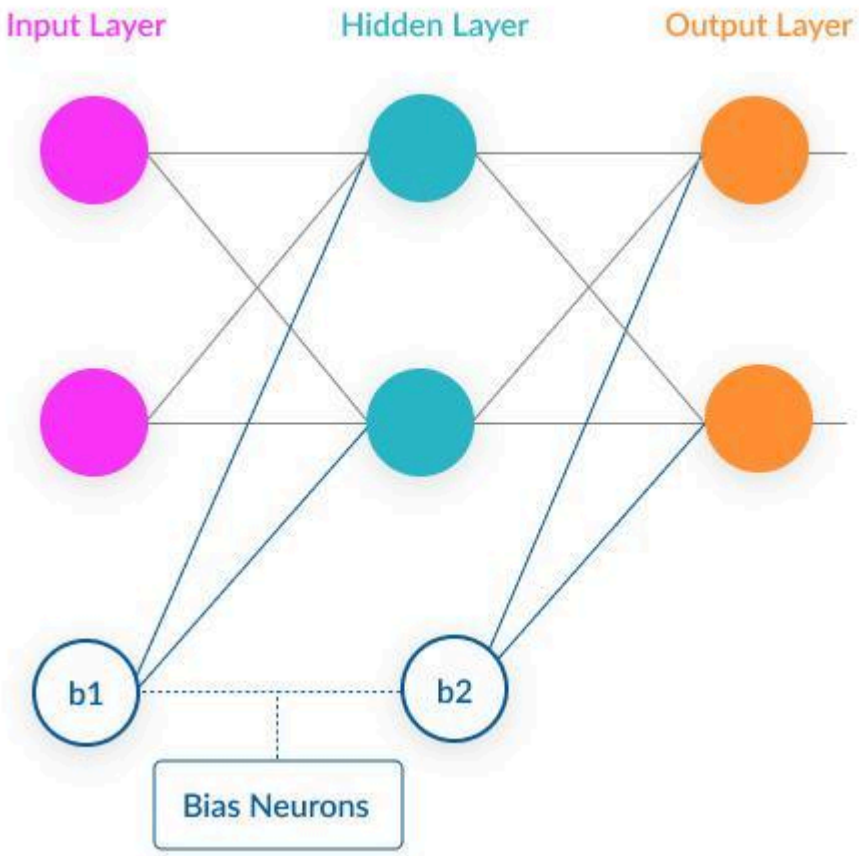




Single-layer perceptron



Multi-layer perceptron



6

Artificial neurons work using **mathematical calculations**.

Formula:

Output = Activation(w1x1 + w2x2 + bias)

Where

Term	Meaning
x	Input
w	Weight
bias	Adjustment value
Activation	Non-linear function

Example:

Input: Study hours, Sleep hours
Neuron → Predict exam score

4. Layers in Neural Networks

A neural network is made of **multiple layers of neurons**.

Structure

Input Layer → Hidden Layer(s) → Output Layer

1️⃣ Input Layer

This layer receives **raw data**.

Example:

Student dataset:

Study Hours	Attendance
5	80%

These values enter the **input layer**.

2️⃣ Hidden Layer

Hidden layers perform **calculations and learn patterns**.

Example:

Input → Hidden Layer → Hidden Layer → Output

More hidden layers = **Deep Learning**

3️⃣ Output Layer

This layer produces the **final prediction**.

Example:

Input	Output
Study Hours	Predicted Marks

Simple Neural Network Example

Study Hours Attendance
↓ ↓
Input Layer
↓
Hidden Layer
↓
Output Layer
Predicted Marks

Why Multiple Layers?

Multiple layers help the model learn **complex patterns**.

Example:

Problem	Layers Needed
Linear prediction	Few layers
Image recognition	Many layers
Speech recognition	Deep networks

Real-World Example

Face Recognition

Image → Neural Network → Detect Face

The network learns features like

- Eyes
- Nose
- Mouth
- Facial structure

AI vs ML vs Deep Learning (Simple ML Example)

This example predicts **student marks based on study hours**.

Using **scikit-learn**

```
from sklearn.linear_model import LinearRegression
import numpy as np

# Study hours
X = np.array([[1],[2],[3],[4],[5]])

# Marks
y = np.array([30,40,50,60,70])

# Model
model = LinearRegression()

# Train
model.fit(X,y)

# Prediction
pred = model.predict([[6]])

print("Predicted Marks:",pred)
```

Output

Predicted Marks: 80

This is **Machine Learning**.

2️⃣ Neural Network Example (Deep Learning)

Now we solve the **same problem using a neural network**.

Using **TensorFlow** and **Keras**

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import numpy as np

# Input data
X = np.array([1,2,3,4,5])
```

```
y = np.array([30,40,50,60,70])

# Build Neural Network
model = Sequential()

model.add(Dense(10, activation='relu', input_shape=(1,)))
model.add(Dense(1))

# Compile
model.compile(optimizer='adam', loss='mse')

# Train model
model.fit(X, y, epochs=200)

# Predict
prediction = model.predict([6])

print("Predicted Marks:", prediction)
```

3️⃣ Artificial Neuron Calculation (Math Example)

This shows how a **single neuron works mathematically**.

```
import numpy as np

# Inputs
x1 = 2
x2 = 3

# Weights
w1 = 0.5
w2 = 0.8

# Bias
b = 1

# Neuron output
z = (w1*x1) + (w2*x2) + b

print("Neuron Output:", z)
```

Formula used

Output = w1x1 + w2x2 + bias

4️⃣ Neural Network Layers Example

Example with **Input Layer → Hidden Layer → Output Layer**

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import numpy as np

# Dataset
X = np.array([[0,0],[0,1],[1,0],[1,1]])
y = np.array([0,1,1,0])

# Model
model = Sequential()

# Hidden layer
model.add(Dense(4, activation='relu', input_shape=(2,)))

# Output layer
model.add(Dense(1, activation='sigmoid'))

# Compile
model.compile(optimizer='adam', loss='binary_crossentropy')

# Train
model.fit(X,y,epochs=500)

# Prediction
print(model.predict([[1,0]]))
```

5️⃣ Deep Learning Example (Digit Recognition)

Dataset: **MNIST Dataset**

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten

# Load data
(X_train,y_train),(X_test,y_test)=mnist.load_data()

# Normalize
X_train = X_train/255
X_test = X_test/255
```

```
# Model
model = Sequential()

model.add(Flatten(input_shape=(28,28)))
model.add(Dense(128,activation='relu'))
model.add(Dense(10,activation='softmax'))

# Compile
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# Train
model.fit(X_train,y_train,epochs=5)

# Evaluate
model.evaluate(X_test,y_test)
```

What Students Learn from These Codes

Code	Concept
Linear Regression	Machine Learning
Neural Network	Deep Learning
Single Neuron	Mathematical concept
Layers Example	Input / Hidden / Output layers
MNIST Example	Real deep learning project

Example Code (Neural Network)

Using **TensorFlow** and **Keras**

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import numpy as np

# Input data
X = np.array([1,2,3,4,5])
y = np.array([30,40,50,60,70])
```

```
# Build Neural Network
model = Sequential()

model.add(Dense(10, activation='relu', input_shape=(1,)))
model.add(Dense(1))

# Compile
model.compile(optimizer='adam', loss='mse')

# Train model
model.fit(X, y, epochs=200)

# Predict
prediction = model.predict([6])

print("Predicted Marks:", prediction)
```

Step-by-Step Explanation

1️⃣ Import Libraries

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import numpy as np
```

Explanation

Library	Purpose
TensorFlow	Deep learning framework
Keras	High-level neural network API
NumPy	Used for numerical data

2️⃣ Create Dataset

```
X = np.array([1,2,3,4,5])
y = np.array([30,40,50,60,70])
```

Explanation

We create **training data**.

Study Hours (X)	Marks (y)
1	30
2	40
3	50
4	60
5	70

The model learns the **relationship between study hours and marks**.

3️⃣ Create Neural Network Model

model = Sequential()

Explanation

`Sequential()` means layers are added **one after another**.

Example structure

Input → Hidden Layer → Output

4️⃣ Add Hidden Layer

model.add(Dense(10, activation='relu', input_shape=(1,)))

Explanation

Parameter	Meaning
Dense	Fully connected neural layer
10	Number of neurons
activation='relu'	Activation function
input_shape=(1,)	One input feature

Structure now:

Input → Hidden Layer (10 neurons)

5️⃣ Add Output Layer

model.add(Dense(1))

Explanation

Output layer has **1 neuron** because we predict **one value (marks)**.

Structure now:

Input → Hidden Layer → Output

6️⃣ Compile Model

model.compile(optimizer='adam', loss='mse')

Explanation

Parameter	Meaning
optimizer='adam'	Algorithm to update weights
loss='mse'	Mean Squared Error

Goal: **Minimize prediction error**.

7️⃣ Train Model

model.fit(X, y, epochs=200)

Explanation

Parameter	Meaning
X	Input data
y	Output labels
epochs	Number of training cycles

During training:

1. Model predicts marks
2. Error is calculated
3. Weights are updated

This repeats **200 times**.

8️⃣ Predict New Value

prediction = model.predict([6])

Explanation

We give **new input**:

Study Hours = 6

Model predicts marks.

9️⃣ Print Output

print("Predicted Marks:", prediction)

Example output:

Predicted Marks: 80

Meaning:
If a student studies **6 hours**, predicted marks **≈ 80**.

Complete Learning Flow

Dataset → Neural Network → Training → Prediction

Example:

Study Hours → Model → Predicted Marks

What Learn from This Code

Concept	Explanation
Dataset	Input data
Neural Network	Model architecture
Training	Learning from data
Loss Function	Error calculation
Prediction	Model output

Deep Learning Architectures

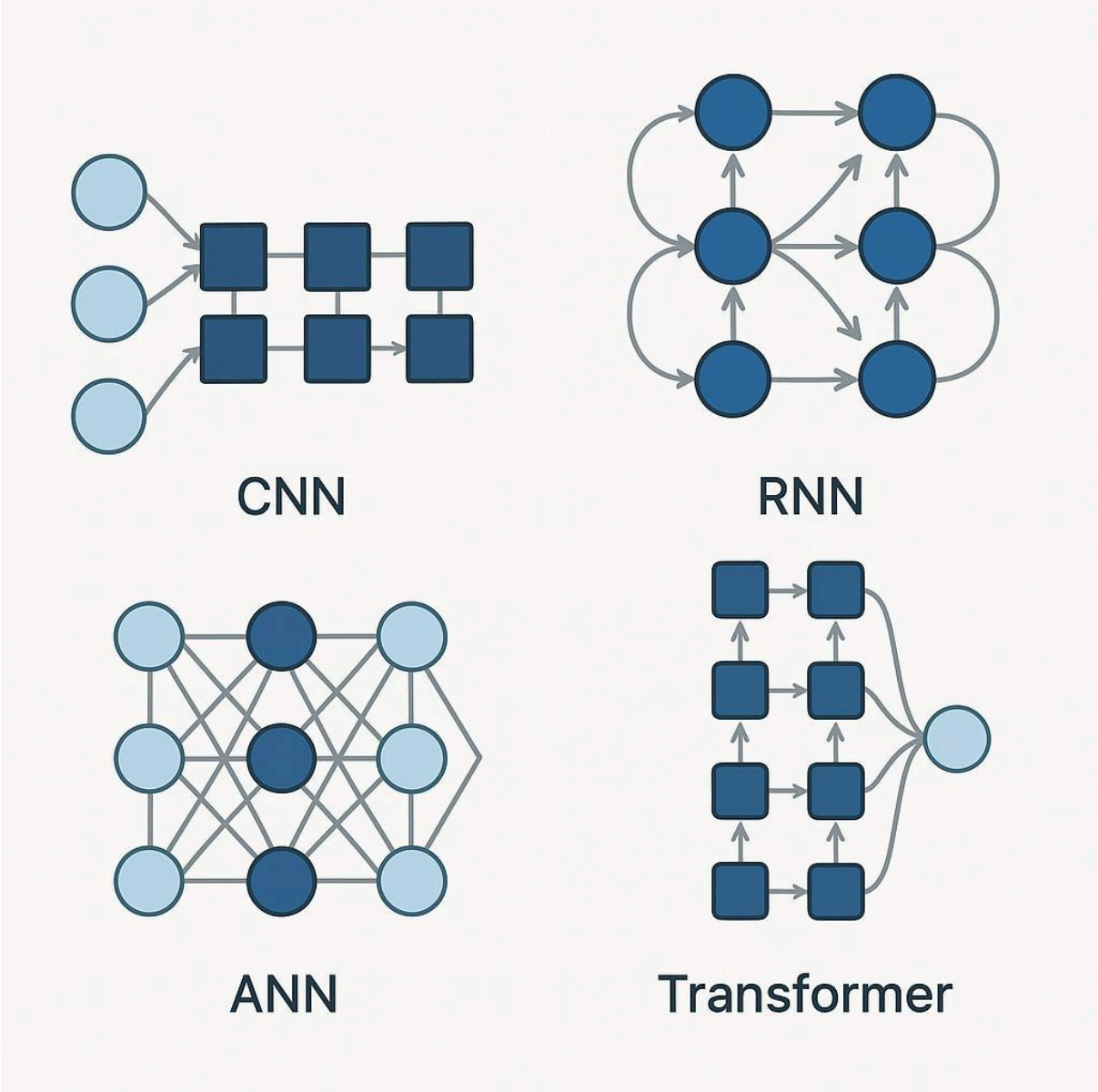
Deep Learning models are built using different **neural network architectures** depending on the **type of data and problem**.

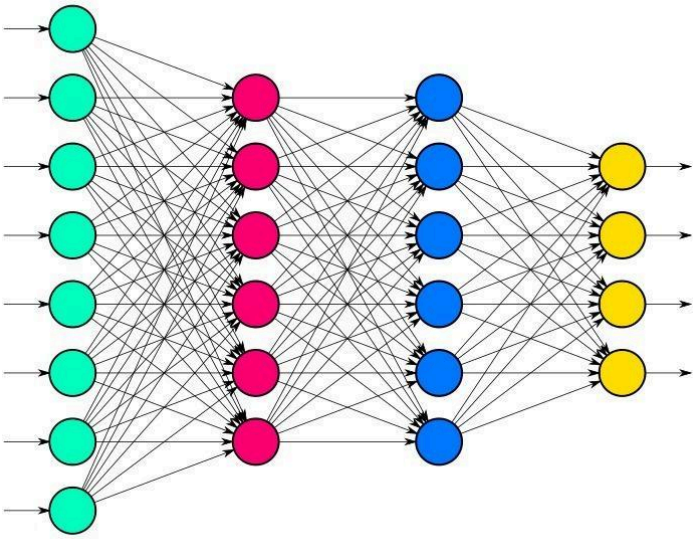
The most common architectures are:

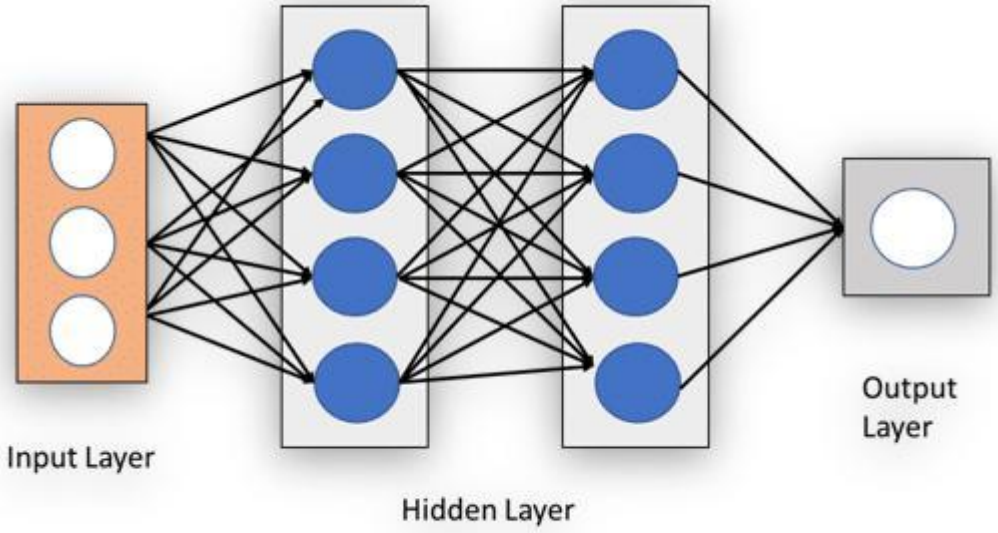
- ANN (Artificial Neural Network)
- CNN (Convolutional Neural Network)
- RNN (Recurrent Neural Network)
- LSTM Networks

These are implemented using libraries like **TensorFlow**, **Keras**, and **PyTorch**.

1 ANN (Artificial Neural Network)







Definition

ANN is the **basic neural network architecture** where every neuron in one layer connects to neurons in the next layer.

Structure

Input Layer → Hidden Layer → Output Layer

Uses

- Sales prediction
- Customer churn prediction

- House price prediction
- Classification problems

Example Python Code

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import numpy as np

# Dataset
X = np.array([[1],[2],[3],[4],[5]])
y = np.array([2,4,6,8,10])

# Model
model = Sequential()
model.add(Dense(10, activation='relu', input_shape=(1,)))
model.add(Dense(1))

model.compile(optimizer='adam', loss='mse')

model.fit(X,y,epochs=200)

print(model.predict([6]))
```

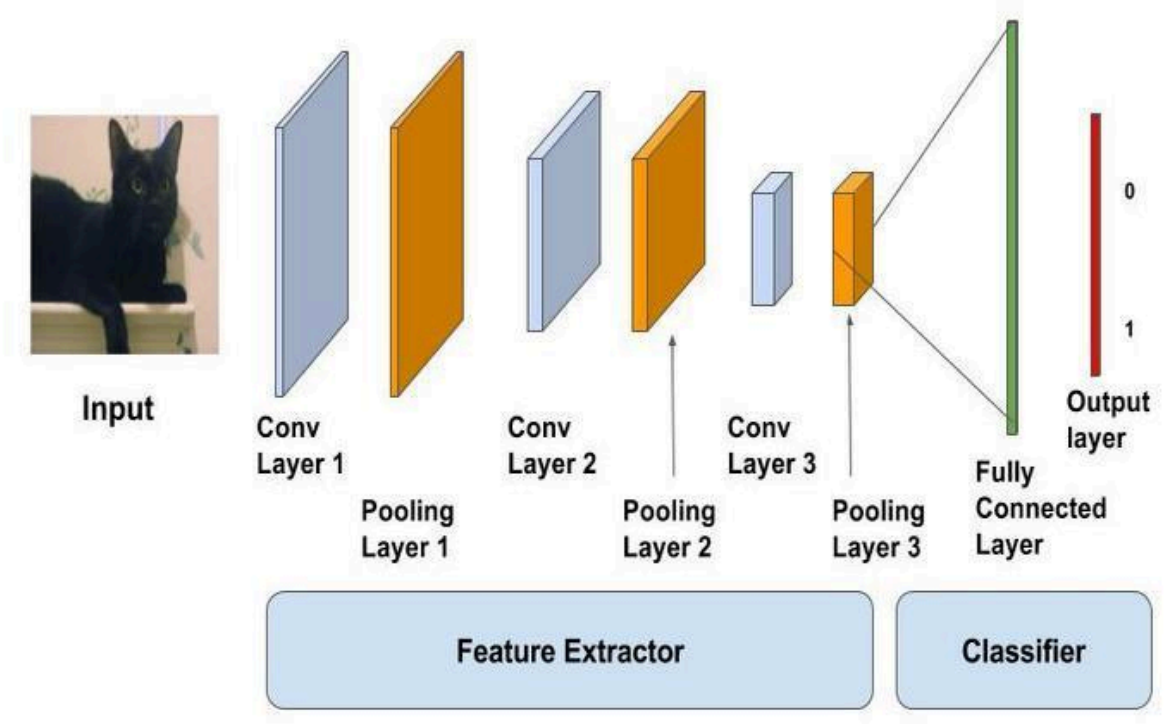
What it does

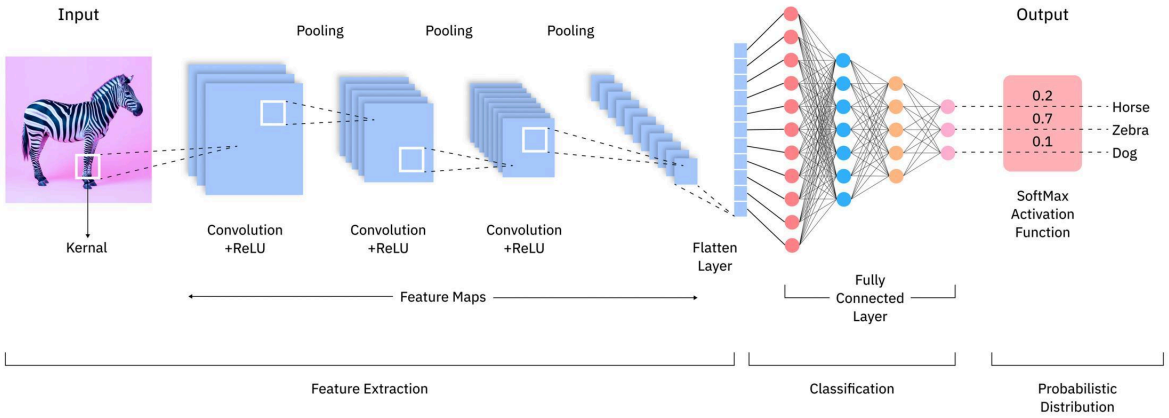
Predicts **output value based on input**.

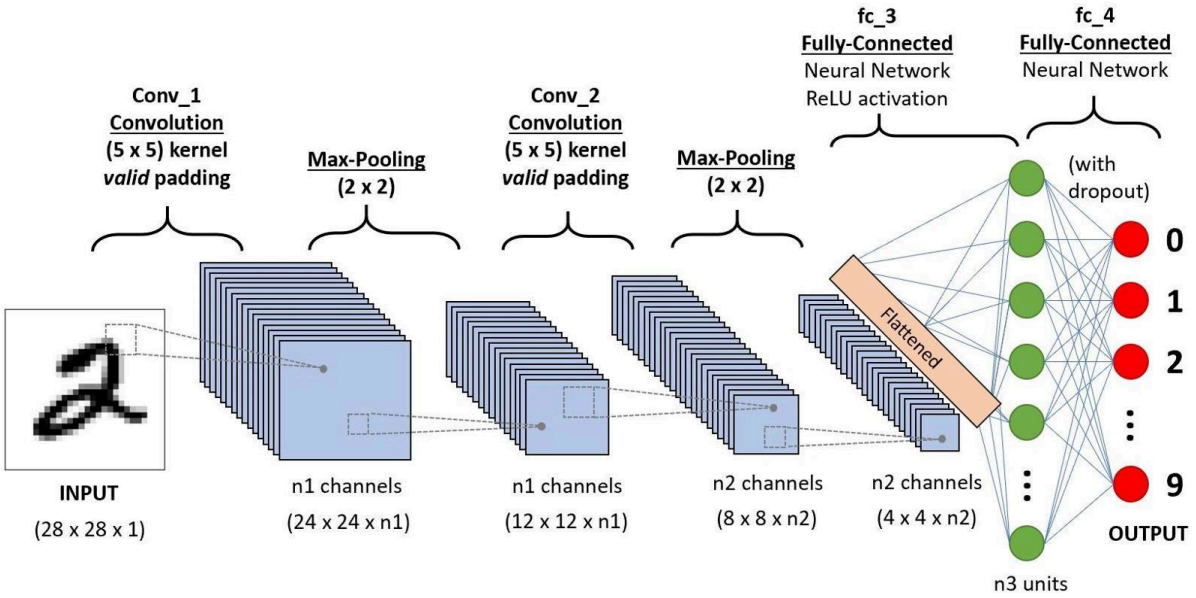
Example

Input = 6
Output ≈ 12

2 CNN (Convolutional Neural Network)







Definition

CNN is used for **image processing and computer vision tasks**.

It automatically detects **patterns like edges, shapes, and objects**.

CNN Layers

Layer	Function
Convolution Layer	Detects features
Pooling Layer	Reduces image size
Fully Connected Layer	Final prediction

Uses

- Face recognition
- Medical image diagnosis
- Self-driving cars
- Object detection

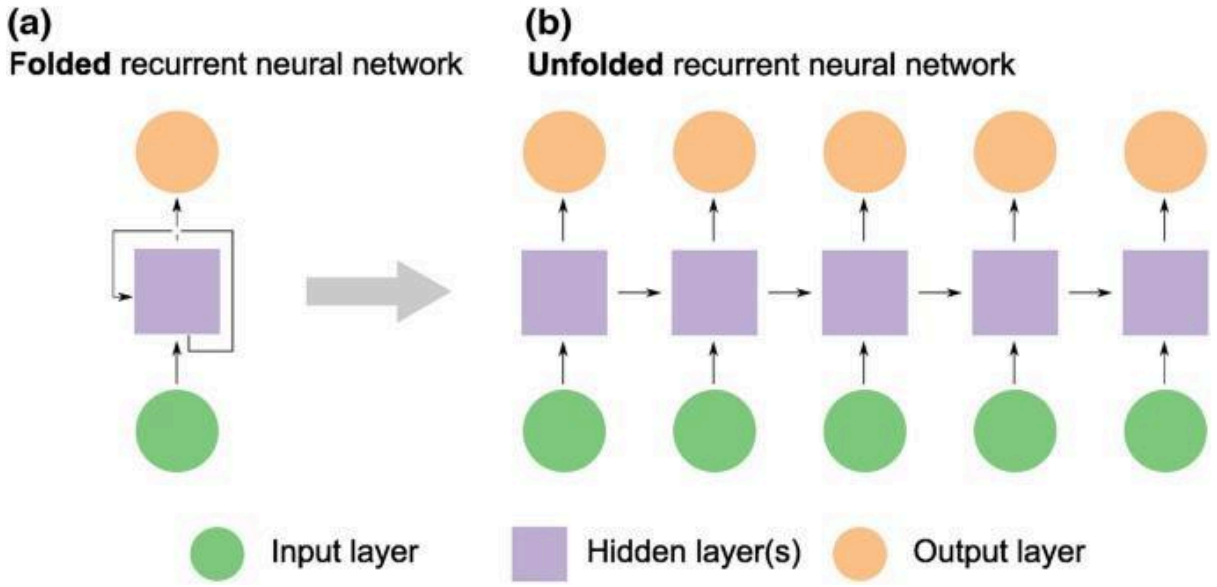
Example Python Code

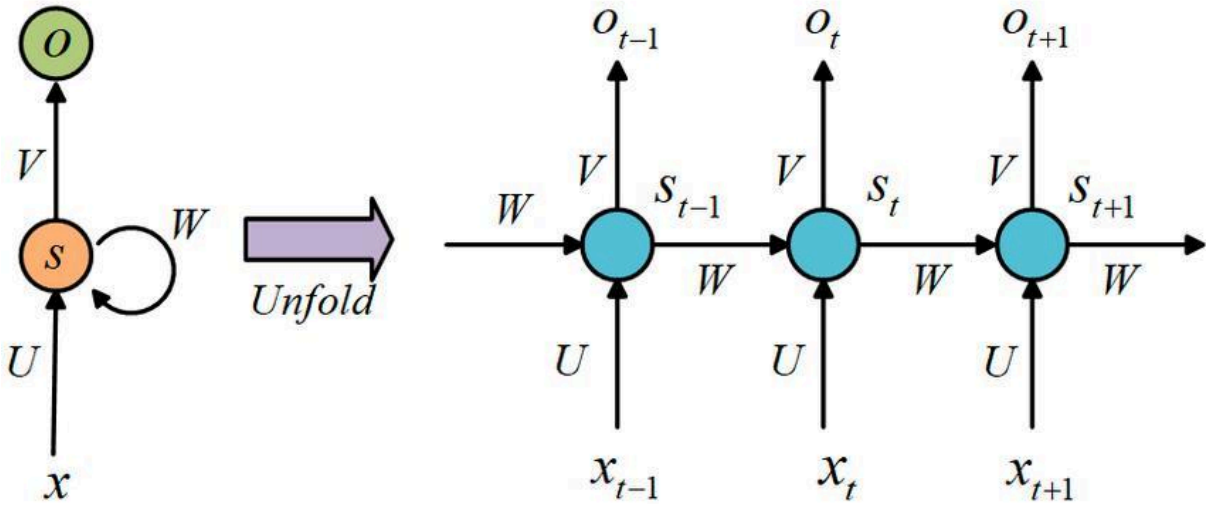
Dataset: **MNIST Dataset**

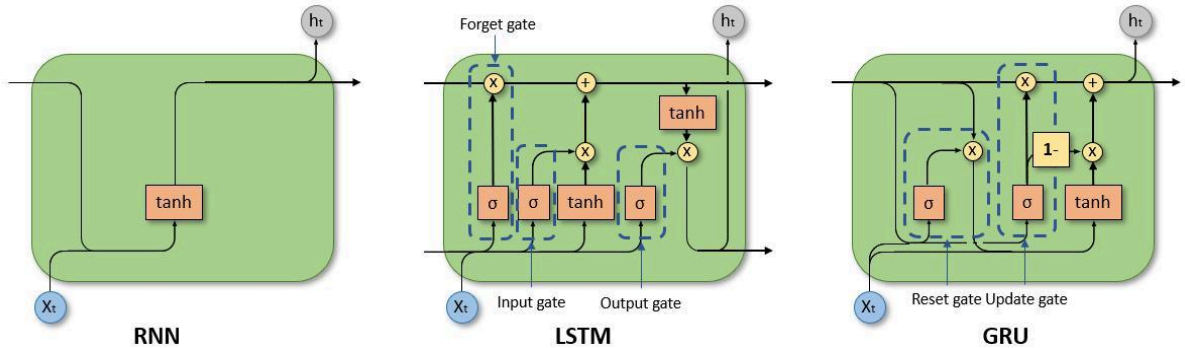
```
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
```

```
(X_train,y_train),(X_test,y_test) = mnist.load_data()

X_train = X_train.reshape(-1,28,28,1)/255
X_test = X_test.reshape(-1,28,28,1)/255

model = Sequential()

model.add(Conv2D(32,(3,3),activation='relu',input_shape=(28,28,1)))
model.add(MaxPooling2D((2,2)))

model.add(Flatten())
model.add(Dense(128,activation='relu'))
model.add(Dense(10,activation='softmax'))

model.compile(optimizer='adam',
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy'])

model.fit(X_train,y_train,epochs=5)
```

3 RNN (Recurrent Neural Network)







Definition

RNN is designed for **sequential data** where previous information is important.

Example sequences:

- Sentences
- Stock prices
- Speech signals

Key Idea

RNN has **memory of previous inputs**.

Example

Word1 → Word2 → Word3 → Prediction

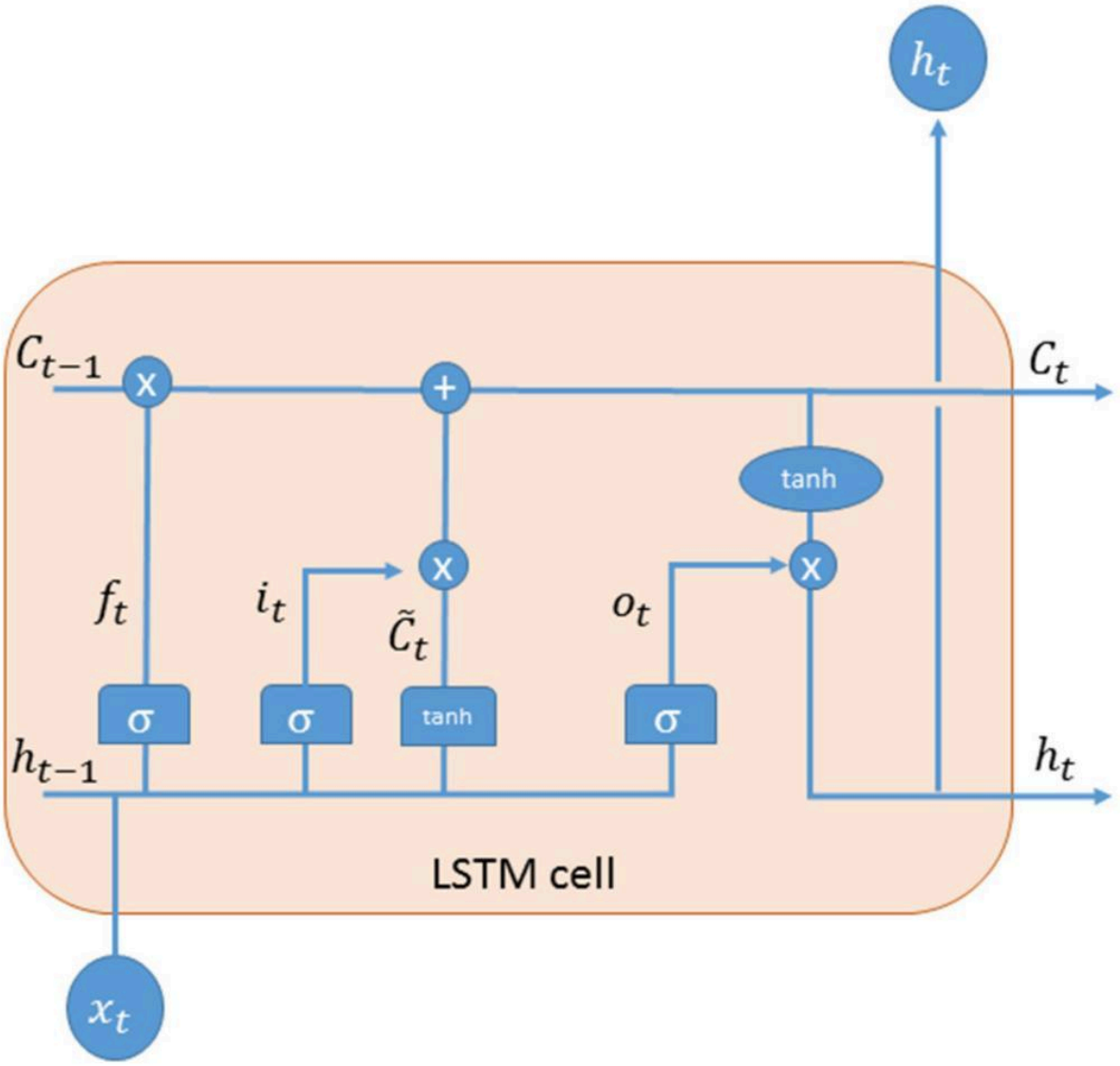
Uses

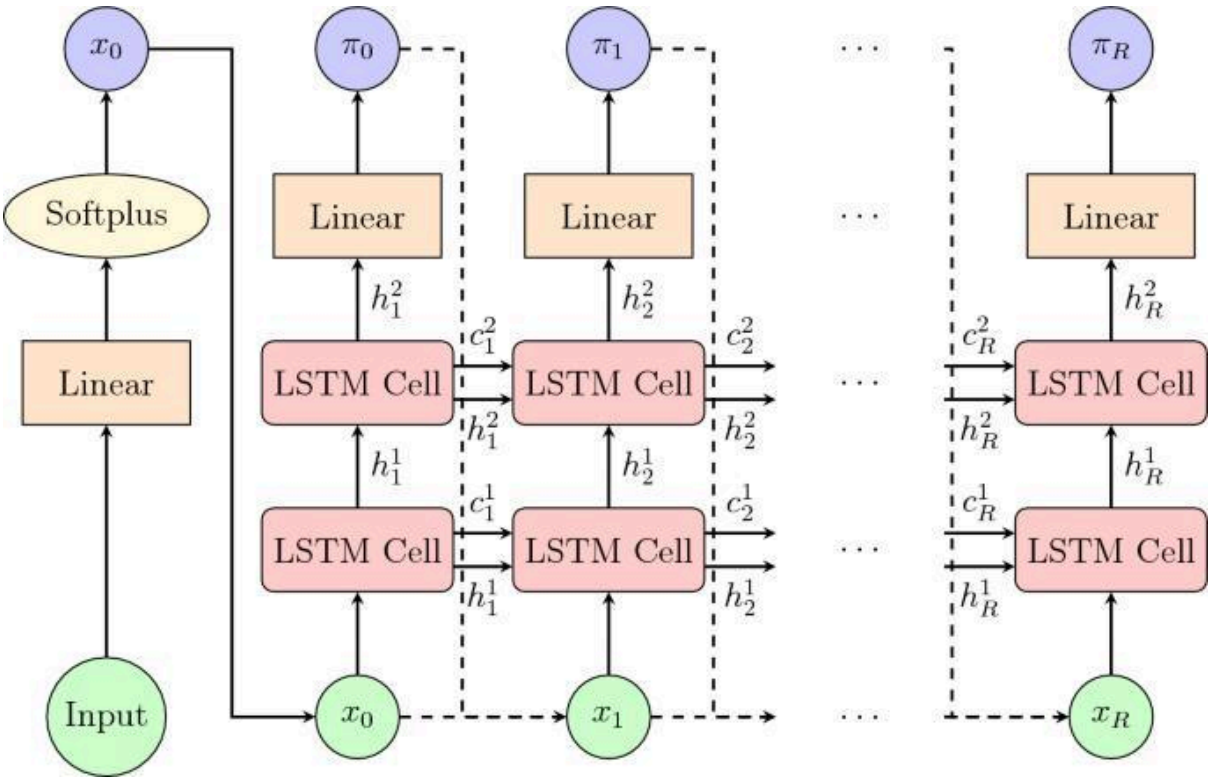
- Language translation
- Speech recognition
- Text generation
- Time series forecasting

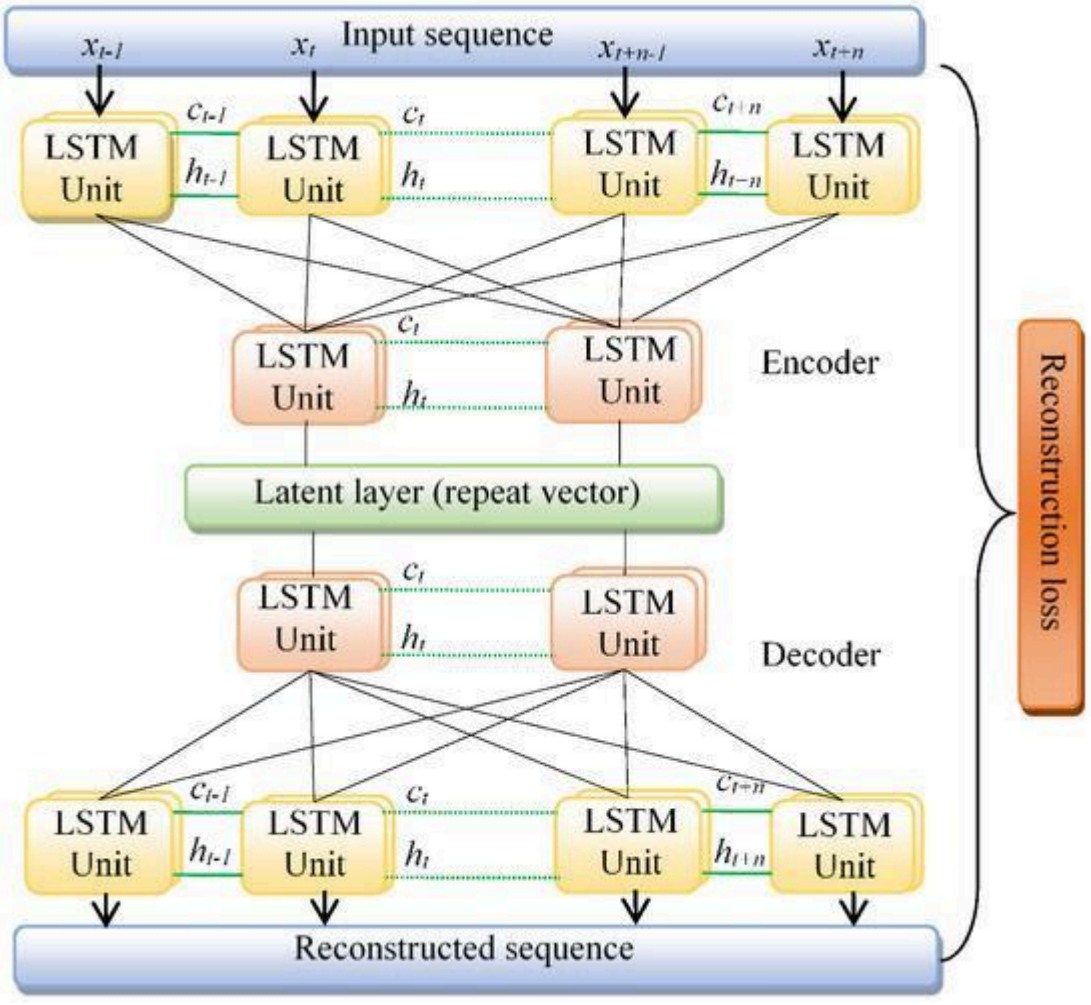
Example Python Code

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, Dense
import numpy as np

# Dummy sequence data
X = np.random.rand(100,5,1)
y = np.random.rand(100,1)

model = Sequential()

model.add(SimpleRNN(10,input_shape=(5,1)))
model.add(Dense(1))

model.compile(optimizer='adam',loss='mse')

model.fit(X,y,epochs=10)
```

4 LSTM Networks




x_0
π_0
π_1
π_R
Softplus
Linear
Linear
Linear
Linear
h_1^2
h_2^2
h_R^2
c_1^2
c_2^2
c_R^2
LSTM Cell
LSTM Cell
LSTM Cell
h_1^2
h_2^2
h_R^2
h_1^1
h_2^1
h_R^1
c_1^1
c_2^1
c_R^1
LSTM Cell
LSTM Cell
LSTM Cell
h_1^1
h_2^1
h_R^1
Input
x_0
x_1
x_R


Input sequence
x_{t-1}
x_t
x_{t+n-1}
x_{t+n}
c_{t-1}
c_t
c_{t+n}
LSTM Unit
h_{t-1}
h_t
h_{t-n}
Encoder
Latent layer (repeat vector)
Decoder
Reconstruction loss
Reconstructed sequence

Definition

LSTM (Long Short-Term Memory) is a **special type of RNN** designed to remember information for **long time periods**.

RNN has a problem called **vanishing gradient**.
LSTM solves this using **gates**.

LSTM Gates

Gate	Function
Forget Gate	Removes irrelevant information
Input Gate	Stores new information
Output Gate	Produces final output

Uses

- Stock price prediction
- Weather forecasting
- Chatbots
- Speech recognition

Example Python Code

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
import numpy as np

# Sample time series data
X = np.random.rand(100,10,1)
y = np.random.rand(100,1)

model = Sequential()

model.add(LSTM(50,input_shape=(10,1)))
model.add(Dense(1))

model.compile(optimizer='adam',loss='mse')

model.fit(X,y,epochs=10)
```

Comparison of Architectures

Model	Best For	Example
ANN	Structured data	Sales prediction
CNN	Image data	Face detection
RNN	Sequence data	Text generation
LSTM	Long sequence data	Stock prediction

1️⃣ ANN (Artificial Neural Network) Code Explanation

Code

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import numpy as np

X = np.array([[1],[2],[3],[4],[5]])
y = np.array([2,4,6,8,10])

model = Sequential()

model.add(Dense(10, activation='relu', input_shape=(1,)))
model.add(Dense(1))

model.compile(optimizer='adam', loss='mse')

model.fit(X,y,epochs=200)

print(model.predict([6]))
```

Using **TensorFlow** and **Keras**

Step 1 — Import Libraries

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import numpy as np
```

Library	Purpose
TensorFlow	Deep learning framework
Keras	Build neural networks
NumPy	Handle numerical data

Step 2 — Create Dataset

X = np.array([[1],[2],[3],[4],[5]])
y = np.array([2,4,6,8,10])

Dataset example:

Input (X)	Output (y)
1	2
2	4
3	6
4	8
5	10

The model learns:

y = 2x

Step 3 — Create Neural Network

model = Sequential()

`Sequential()` means layers are **added one after another**.

Structure:

Input → Hidden Layer → Output

Step 4 — Add Hidden Layer

model.add(Dense(10, activation='relu', input_shape=(1,)))

Parameter	Meaning
Dense	Fully connected layer

10	Number of neurons
relu	Activation function
input_shape=(1,)	One input feature

Now architecture:

Input → 10 Neurons

Step 5 — Add Output Layer

model.add(Dense(1))

Only **1 neuron** because output is **one value**.

Final structure:

Input → Hidden Layer → Output

Step 6 — Compile Model

model.compile(optimizer='adam', loss='mse')

Parameter	Meaning
optimizer	Updates model weights
adam	Popular optimizer
loss	Error calculation
mse	Mean Squared Error

Goal: **Minimize error**.

Step 7 — Train Model

model.fit(X,y,epochs=200)

Parameter	Meaning
X	Input data
y	True output
epochs	Number of training cycles

During training:

Prediction → Error → Update weights

Repeated **200 times**.

Step 8 — Predict New Value

print(model.predict([6]))

Input:

6

Output (approx):

12

Model learned the pattern **y = 2x**.

2️⃣ CNN Code Explanation

Code

```
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
```

Dataset: **MNIST Dataset**

Load Dataset

```
(X_train,y_train),(X_test,y_test) = mnist.load_data()
```

Loads **handwritten digit images**.

Image size:

28 × 28 pixels

Normalize Data

```
X_train = X_train.reshape(-1,28,28,1)/255
```

Why?

Pixel values: 0 – 255

Normalize:

0 – 1

Helps model learn faster.

Convolution Layer

model.add(Conv2D(32,(3,3),activation='relu',input_shape=(28,28,1)))

Meaning:

Parameter	Meaning
32	Filters
(3,3)	Kernel size
relu	Activation

This layer detects:

- edges
- shapes
- patterns

Pooling Layer

model.add(MaxPooling2D((2,2)))

Purpose:

Reduce image size

Benefits:

- faster computation
- reduces overfitting

Flatten Layer

model.add(Flatten())

Converts:

2D Image → 1D vector

Example

28×28 → 784 values

Dense Layer

model.add(Dense(128,activation='relu'))

Fully connected layer that **learns patterns**.

Output Layer

model.add(Dense(10,activation='softmax'))

Why **10 neurons**?

Digits:

0 1 2 3 4 5 6 7 8 9

Softmax outputs **probability for each digit**.

3️⃣ RNN Code Explanation

Code

from tensorflow.keras.layers import SimpleRNN

RNN processes **sequential data**.

Example sequences:

text
time series
speech

Create Sequence Data

X = np.random.rand(100,5,1)

Meaning:

100 samples
5 time steps
1 feature

Example sequence:

[10,12,14,16,18]

RNN Layer

model.add(SimpleRNN(10,input_shape=(5,1)))

Meaning:

Parameter	Meaning
10	Neurons
(5,1)	Sequence length

RNN remembers **previous values**.

---

4 LSTM Code Explanation

Code

from tensorflow.keras.layers import LSTM

LSTM is **advanced RNN**.

Used for:

- stock prediction
- speech recognition
- NLP

LSTM Layer

model.add(LSTM(50,input_shape=(10,1)))

Meaning:

Parameter	Meaning
50	Memory units
(10,1)	Time steps

LSTM remembers **long-term patterns**.

Architecture Summary

Model	Data Type	Example
ANN	Structured data	Price prediction
CNN	Image data	Face recognition
RNN	Sequence data	Text prediction
LSTM	Long sequences	Stock forecasting